

UADC PoC Collaboration Workspace

This workspace is for the xBRL-GL Next / UADC proof of concept. It implements the UADC processing model for "A Hierarchical Tidy Data Universal Audit Data Converter for Invoice Reuse".

The proposal defines UADC as a Universal Audit Data Converter that uses hierarchical tidy data as a common semantic representation. The target architecture separates source syntax conversion from downstream audit-view generation:

- syntax binding maps source invoice XML into a common EN 16931 / XBRL GL Next semantic layer;
- the common semantic layer preserves document-level invoice information, parties, tax subtotals, invoice lines, identifiers, dates, currencies, and monetary amounts;
- semantic binding then projects the common dataset into downstream audit views, including ISO 21378:2019 Audit data collection Sales/Purchase invoice views and AICPA ADS O2C/P2P invoice views.

This repository is the working implementation space for that proposal. The current implementation completes the planned Phase 1 and Phase 2 PoC baseline. Phase 1 provides the EN 16931 Structured CSV, xBRL-CSV taxonomy and metadata, validation, and UBL round trip. Phase 2 provides ADS XBRL GL and ADS PSV target views plus ISO 21378 ADC invoice CSV views with explicit mapping-gap classification.

ADS and ISO 21378 ADC assume a wider ERP environment in which user activity history is captured and detailed accounting records continue into period-end processing and annual-report preparation. An invoice alone therefore cannot populate every audit-data field. This PoC makes that boundary explicit: it shows which audit views can be derived from invoice data, which fields require transformation or semantic approximation, and which fields must be supplied by ERP ledgers, master data, workflow logs, or other operational systems.

Source and target interface files may remain organization-specific. By binding those interfaces to the common UADC semantic structure, an organization can separate changing application formats from the meaning, hierarchy, provenance, and validation rules of the retained data. The PoC therefore demonstrates a practical foundation for stable, long-term data retention that remains traceable and usable in audits, while allowing ERP products and interface formats to change over time.

The first PoC checkpoint is the EN 16931-1 invoice semantic model represented as an LHM/HMD-style CSV, plus binding-driven conversion from UBL Invoice XML to structured CSV, xBRL-CSV JSON metadata, and round-trip UBL Invoice XML.

OpenPeppol BIS Billing is handled as the next layer: a CIUS/profile overlay on top of EN 16931-1, with additional constraints, defaults, and syntax-specific rules.

Clone And Setup Overview

After cloning the GitHub repository, use this sequence to prepare a local execution and development environment.

Windows PowerShell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

macOS / Linux shell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

In PowerShell, **\$python = 'python'** is a variable assignment and **& \$python ...** uses PowerShell's call operator to run that command. In macOS/Linux shells, use **PYTHON=python3** and run commands as **\$PYTHON ...** without **&**. If Python is not on **PATH**, set the variable to the full local Python executable path instead.

If this repository is checked out inside a larger workspace, enter the PoC directory instead:

```
cd <local-clone-directory>\UADC-PoC
```

Then prepare generated local files and test artifacts:

Windows PowerShell:

```
& $python -m py_compile .\src\syntax_binding.py
.\tools\build_roundtrip_test_artifacts.py
.\tools\taxonomy\xBRLGL_TaxonomyGenerator.py
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tools\build_roundtrip_test_artifacts.py
& $python .\tests\test_roundtrip_artifacts.py
```

macOS / Linux shell:

```
$PYTHON -m py_compile ./src/syntax_binding.py
./tools/build_roundtrip_test_artifacts.py
./tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

```
$PYTHON ./tests/test_xbrlgl_generator_uadc_lhm.py
$PYTHON ./tools/build_roundtrip_test_artifacts.py
$PYTHON ./tests/test_roundtrip_artifacts.py
```

This creates the local xBRL-CSV taxonomy under **out/taxonomy/** and refreshes reviewable round-trip artifacts under **tests/roundtrip/**.

For the full initial setup and continuing development test workflow, see **docs/development/README.md**. For conversion internals and day-to-day test commands, see **docs/README_SCRIPT_PROCESSING.md** and **docs/testing/README.md**.

Figure 1

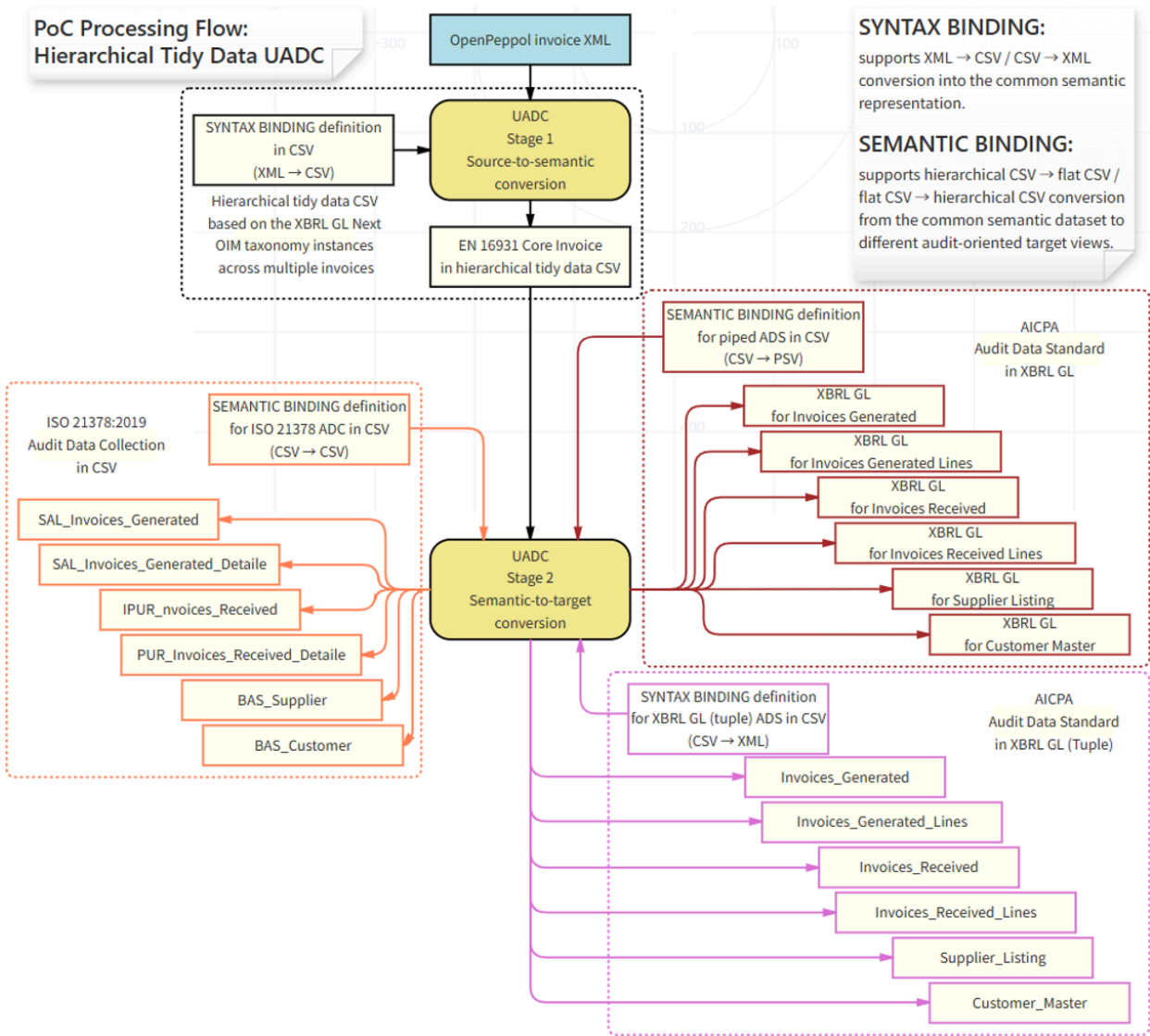


Figure 1 from the proposal shows the intended UADC processing flow. Phase 1 covers the source-to-semantic path from OpenPeppol/UBL Invoice XML to EN 16931 hierarchical tidy data, structured CSV, JSON metadata, taxonomy validation, and round-trip XML. Phase 2 extends the same semantic layer toward ISO 21378 and AICPA ADS target views through semantic binding.

UADC Processing Steps

The phases below describe the UADC processing model, not project management milestones.

Phase	Processing Step	Main Inputs And Outputs	Current Status
Phase 1	Create a generic Structured CSV from source invoice syntaxes.	Input starts with Peppol UBL Invoice XML. The source syntax binding maps invoice facts into the common EN 16931 / UADC hierarchical Structured CSV. The same phase writes xBRL-CSV metadata JSON and validates the generic representation against the generated xBRL-CSV taxonomy.	Complete for the PoC baseline. Peppol UBL input, Structured CSV generation, metadata JSON generation, taxonomy and metadata validation, and UBL round-trip schema checks are functioning.
Phase 2	Convert the generic Structured CSV into purpose-specific common formats.	The current targets are six ADS XBRL GL views, six ADS PSV views, and ISO 21378:2019 ADC Tables 38, 39, 53, and 54 CSV views.	Complete for the planned PoC scope. Target bindings, generation, regression tests, and documented ISO mapping gaps are in place.
Phase 3	Expand supported input syntaxes and interoperability tests.	Add UN/CEFACT Invoice and XBRL GL invoice examples as additional source inputs, alongside Peppol UBL. Add corresponding output conversions to Peppol Invoice, UN/CEFACT Invoice, and XBRL GL invoice.	Future direction. The later test target includes XBRL GL profiles being discussed in Finland and Estonia.

Phase 1 intentionally focuses on the neutral intermediate representation: a generic, hierarchical Structured CSV that can be validated and round-tripped. Phase 2 uses that common representation as the source for multiple downstream formats. This separation is the core UADC idea: source syntax conversion is kept separate from target-format projection.

Processing Flow

Phase 1 source syntax conversion

Peppol UBL Invoice XML

future: UN/CEFACT Invoice, XBRL GL invoice

-> syntax binding

-> generic EN 16931 / UADC Structured CSV

-> xBRL-CSV metadata JSON

-> taxonomy and metadata validation

-> round-trip validation back to source syntax where supported

Phase 2 target-format projection

```
generic EN 16931 / UADC Structured CSV
-> ADS XBRL GL syntax binding
  -> Invoices Received/Generated and Lines
  -> Supplier Listing and Customer Master
-> ADS semantic binding
  -> ADS O2C/P2P PSV views
-> ISO 21378 semantic binding
  -> ADC Tables 38, 39, 53, and 54 CSV views
```

Future additional target projections

```
-> Peppol Invoice
-> UN/CEFACT Invoice
-> other XBRL GL invoice profiles
```

Later interoperability tests

```
-> XBRL GL invoice examples and profiles
-> Finland and Estonia XBRL GL discussion targets
```

Directory Layout

- **docs/** - Human-readable project documentation. Start with **docs/README.md**. Script processing is explained in **docs/README_SCRIPT_PROCESSING.md**; binding contracts are documented in **docs/syntax_binding_conversion/** and **docs/semantic_binding_conversion/**; specification CSV roles in **docs/specifications/README.md**; setup and tool usage in **docs/development/README.md**; test and round-trip procedures in **docs/testing/README.md**; sample and reference files in **docs/repository_files/README.md**; and design decisions in **docs/decisions/**.
- **references/** - External source notes and links used to interpret standards, source specifications, and implementation references. Keep large licensed source documents outside the repository and record only reproducible notes or pointers here.
- **specs/lhm/** - LHM/HMD semantic model definitions for the EN 16931 invoice PoC. The generated/current CSV is stored here, while **specs/lhm/source/** keeps the editable source CSV used to regenerate or adjust the LHM. Local reviewer workbooks are ignored by Git.
- **specs/bindings/** - Binding definitions. The active UBL Invoice syntax binding is **specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv**; it maps LHM semantic paths to UBL XPath expressions and selector predicates used by forward and reverse conversion. Phase 2 ADS XBRL GL binding CSV files are under **specs/bindings/syntax/**. The review workbook is **specs/bindings/ADS_XBRL_GL_Bindings.xlsx**. ADS PSV and ISO 21378 ADC CSV bindings are under **specs/bindings/semantic/**.
- **samples/input/** - Small sample input files committed for baseline checks, including the minimal UBL Invoice sample and selected BIS Billing example invoices.
- **samples/expected/** - Checked expected output for lightweight regression checks where a stable expected artifact is useful.
- **src/** - Operational conversion scripts and beginner tutorial wrappers. The main converter is **src/syntax_binding.py**; it generates hierarchical Structured CSV, writes xBRL-CSV metadata

JSON, and performs Structured-CSV-to-XML round trips. **src/syntax_binding_ads_xbri_gl.py** generates the Phase 2 ADS XBRL GL target instances, and **src/tutorial/** provides simple learning scripts.

- **tests/** - Regression checks and generated round-trip review artifacts. **tests/roundtrip/** keeps source XML, structured CSV, xBRL-CSV metadata JSON, and regenerated XML side by side for review.
- **tools/** - Initial setup, supporting generation, environment-maintenance, and development helper tools. This includes the round-trip artifact builder **tools/build_roundtrip_test_artifacts.py**, tutorial/sample converters, LHM and binding maintenance scripts, and the local taxonomy generator in **tools/taxonomy/**. All 15 programs are specified in **docs/tools/program_specification.md**.
- **out/** - Generated PoC evidence and target output tracked by Git. This includes taxonomy output, Structured CSV output, reverse-conversion output, Phase 2 target views, and selected rendered document QA artifacts. Regenerate these files from their source definitions rather than editing them directly.

The taxonomy generator is included at **tools/taxonomy/xBRLGL_TaxonomyGenerator.py**. The generated xBRL-CSV taxonomy entry point is **out/taxonomy/plt/en16931-oim-2026-07-05.xsd** and its dimensional definition linkbase is **out/taxonomy/plt/en16931-def-2026-07-05.xml**. The entry point also discovers the EN 16931 presentation linkbase so Arelle displays the LHM hierarchy. Tuple/content taxonomy schemas such as **plt-all-.xsd** and **en16931-content-.xsd** are not generated for this PoC.

Current Scope

1. Define and audit the EN 16931 invoice LHM used by the generic Structured CSV.
2. Convert Peppol UBL Invoice XML into the generic UADC Structured CSV.
3. Generate xBRL-CSV metadata that references the **en16931-oim** taxonomy entry point.
4. Validate generated xBRL-CSV metadata with Arelle.
5. Reconstruct UBL Invoice XML from Structured CSV and validate it with UBL 2.1 schemas.
6. Generate Phase 2 ADS XBRL GL target views from the Phase 1 Structured CSV. The current ADS XBRL GL outputs are written under **out/phase2/ADS_XBRL_GL//** as **Invoices_Received.xbri**, **Invoices_Generated.xbri**, **Invoices_Received_Lines.xbri**, **Invoices_Generated_Lines.xbri**, **Supplier_Listing.xbri**, and **Customer_Master.xbri**.
7. Generate Phase 2 ADS PSV views under **out/phase2/ADS_PSV/** and ISO 21378 ADC invoice CSV views under **out/phase2/ISO21378_ADC/**.
8. Keep OpenPeppol BIS Billing as the first CIUS/profile layer on top of the EN 16931 baseline.
9. Prepare the next-phase design for additional source inputs and target profiles, including UN/CEFACT Invoice and other XBRL GL invoice examples.

Tasks

1. Define the LHM for the generic Structured CSV. The LHM describes the EN 16931 invoice business terms as a hierarchy. It includes semantic paths, effective **lhm_level** values, and syntax binding references. The first syntax binding maps Peppol UBL Invoice XML into this neutral Structured CSV representation.

2. Define the xBRL-CSV taxonomy for the LHM. The taxonomy generator reads the LHM CSV and emits the EN 16931 module schema, **en16931-oim** entry point, **en16931-def** dimensional definition linkbase, labels, and presentation linkbase. Hypercubes, dimensions, primary items, and presentation parent-child relationships are derived from the effective LHM hierarchy. Class/BG presentation nodes use OIM primary items; BT/fact nodes use module concepts. The generated taxonomy is checked with Arelle using **out/taxonomy/plt/en16931-oim-2026-07-05.xsd** as the entry point.
3. Convert XML instances to structured CSV with JSON metadata. The syntax binding converter reads a Peppol UBL Invoice XML instance, applies the LHM XPath bindings, and writes a hierarchical Structured CSV. At the same time it writes xBRL-CSV JSON metadata that links CSV dimensions and fact columns to the generated taxonomy. The JSON metadata is validated with Arelle **loadFromOIM**.
4. Perform round trip conversion from structured CSV to XML instances. The reverse conversion reads the structured CSV and the same LHM/syntax binding definitions, reconstructs a UBL Invoice XML instance, and adds required syntax support values where needed for UBL schema validity. Absolute binding XPaths that point outside a repeated semantic context remain rooted at the UBL document; for example, BT-90 is semantically under payment instructions but is written to **AccountingSupplierParty**, not below **PaymentMeans**. The resulting XML is checked with an XML parser and UBL 2.1 schema validation.
5. Add additional source syntaxes after the Peppol UBL baseline is stable. Planned source inputs include UN/CEFACT Invoice and XBRL GL invoice examples. These should map into the same generic Structured CSV wherever the semantic model overlaps.
6. Complete the planned Phase 2 target projections from the generic Structured CSV. ADS O2C/P2P XBRL GL and PSV bindings and ISO 21378:2019 ADC Tables 38, 39, 53, and 54 CSV bindings are defined and tested. Supplier Listing places Seller postal address under the **identifierType=V** identifier reference. The ISO bindings identify direct mappings, semantic approximations, required transformations, and data absent from EN 16931. This completes Phase 2 for the PoC scope; it does not claim that EN 16931 alone supplies every ISO 21378 audit field.

The resulting artifacts are therefore checked at two levels: xBRL-CSV reports and taxonomy are verified with Arelle, while regenerated XML instances are verified with XML parsing and UBL schema validation.

Current Tests

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
& $python .\tests\test_openpeppol_invoice_conversion.py
& $python .\tests\test_bis_billing3_examples_conversion.py
& $python .\tests\test_roundtrip_artifacts.py
& $python .\tests\test_xbrl_csv_metadata_arelle.py
& $python .\tests\test_ads_invoices_received_xbrl_gl.py
```

```
& $python .\tests\test_ads_invoices_generated_xbrl_gl.py  
& $python .\tests\test_ads_invoices_received_lines_xbrl_gl.py  
& $python .\tests\test_ads_invoices_generated_lines_xbrl_gl.py  
& $python .\tests\test_ads_supplier_listing_xbrl_gl.py  
& $python .\tests\test_ads_customer_master_xbrl_gl.py
```

The sample UBL Invoice XML and BIS Billing examples are converted using **specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv**. OpenPeppol CIUS checks are added after the EN 16931 conversion baseline is stable.

Semantic path elements are generated from Business Terms using **lowerCamelCaseConcatenated**, for example:

```
Invoice issue date -> invoiceIssueDate  
Seller postal address -> sellerPostalAddress
```